

DESIGN AND DEVELOP A REACT JS BASED WEB APPLICATION FOR TICKET BOOKING OF AN INTERNAL DEPARTMENT EVENT

School of Computing

**Bachelor of Technology
in
Computer Science & Engineering
By**

J HIMA VARSHITHA (VTU27179)

*10211CS224
Full Stack Application Development
Slot : E2*



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING**

**VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D
INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University Estd u/s 3 of UGC Act, 1956)
CHENNAI 600 062, TAMILNADU, INDIA**

April, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled "Design and develop a React JS based web application for Ticket Booking of an Internal Department Event" by J HIMA VARSHITHA (VTU27179) has been carried out in Winter semester of Academic year 2025-2026(WS2526).

Signature of Faculty

Dr.Chidambaranathan C M.

Assistant Professor

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

April, 2026

Signature of Course Co-ordinator of the Department

Dr. M. Sumithra

Assistant Professor(Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

April, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others ideas or words have been included, we have adequately cited and referenced the original sources. I also declare that i have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

J. Hima varshitha

Date: 06/04/2026

ABSTRACT

In recent years, digital transformation has significantly improved the management of academic and departmental activities. Traditional methods of event registration, such as manual entries and offline coordination, are often inefficient, time-consuming, and prone to errors. To overcome these challenges, this project focuses on the design and development of a React JS based web application for ticket booking of internal department events.

The application provides a user-friendly interface that allows users to browse available events, view detailed information, and book tickets seamlessly. The frontend is developed using React JS to ensure a dynamic and responsive user experience, while the backend is implemented using Node.js and Express.js to handle API requests and manage application logic. MongoDB Atlas is used as a cloud-based database to store user and booking information securely.

The system follows a client-server architecture, where the frontend communicates with the backend through RESTful APIs. Users can perform actions such as event selection, ticket booking, and payment simulation. The application also supports basic authentication and maintains booking records for future reference.

In conclusion, the developed system demonstrates the effective integration of modern web technologies to build a scalable and efficient full-stack application. Furthermore, it provides a strong foundation for future enhancements such as real-time notifications, secure payment integration, and advanced user authentication.

Keywords: React JS, Node.js, Express.js, MongoDB Atlas, Event Ticket Booking, Web Application, REST API, Full Stack Development, Cloud Database

LIST OF FIGURES

1.1	Framework Diagram	5
4.1	Architecture Diagram	18
4.2	Data Flow Diagram	19
5.1	Framework Diagram	28
5.2	User Authentication	29
5.3	Event Listing Management	29
5.4	Framework Diagram	30
5.5	Framework Diagram	31
5.6	User Dashboard	31
5.7	Ticket	32

LIST OF TABLES

2.1	Comparative Analysis of Market Alternatives	7
7.1	Mapping of the Project to Complex Engineering Problem (CEP) . .	36
7.2	Mapping of the Project to Complex Engineering Activities (CEA) .	37
7.3	SDG Mapping for the Project	38

LIST OF ACRONYMS AND ABBREVIATIONS

API	Application Programming Interface
CI	Continuous Integration
DB	Database
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
ORM	Object Relational Mapping
REST	Representational State Transfer
UI	User Interface
UX	User Experience

TABLE OF CONTENTS

	Page.No
ABSTRACT	iii
LIST OF FIGURES	iv
LIST OF TABLES	v
LIST OF ACRONYMS AND ABBREVIATIONS	vi
1 INTRODUCTION	1
1.1 Introduction	1
1.2 Aim of the Project	2
1.3 Scope of the Project	3
1.4 Framework	4
2 SURVEY OF SIMILAR APPLICATION IN MARKET	6
3 FRAMEWORK DESCRIPTION	8
3.1 Frontend Implementation	8
3.2 Backend Implementation	11
3.3 Database Implementation	13
4 METHODOLOGIES	17

4.1	Project Architecture	17
4.2	DataFlow Diagram	19
4.3	Module 1: User Management	21
4.4	Module 2: Event Management	21
4.5	Module 3: Booking and Payment Management	22
5	RESULTS AND DISCUSSIONS	25
5.1	System Modules and Features	25
5.2	System Performance and Discussion	26
5.3	System Interface Visualizations	28
5.3.1	Landing Page Interface	28
5.3.2	User Authentication:	28
5.3.3	Event Listing Management	29
5.3.4	Ticket Booking System	30
5.3.5	Booking Details	30
5.3.6	User Dashboard	31
5.3.7	Ticket Generation	32
6	CONCLUSION AND FUTURE ENHANCEMENTS	33
6.1	Conclusion	33
6.2	Future Enhancements	34
7	ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG	35
7.1	Mapping of the Project to Complex Engineering Problem (CEP) . .	35
7.2	Mapping of the Project to Complex Engineering Activities (CEA) .	37
7.3	SDG Mapping (CEP Section)	38
	References	38

Chapter 1

INTRODUCTION

1.1 Introduction

In the modern era, digital technologies have transformed the way organizations manage and conduct events. Educational institutions and departments frequently organize various internal events such as workshops, seminars, technical fests, and cultural programs. Traditionally, the registration and ticket booking process for these events has been carried out manually, which often leads to inefficiencies, data inconsistencies, and increased administrative workload.

To address these challenges, this project focuses on the design and development of a React JS based web application for ticket booking of internal department events. The proposed system provides a centralized platform where users can easily browse available events, view detailed information, and book tickets in a convenient and efficient manner. By automating the booking process, the application minimizes manual effort and enhances the overall user experience.

The frontend of the application is developed using React JS, which enables the creation of a dynamic and responsive user interface. The backend is implemented using Node.js and Express.js, which handle server-side operations and API communication. MongoDB Atlas is used as a cloud-based database to securely store user

and booking data. The integration of these technologies ensures a scalable, flexible, and high-performance system.

The application follows a client-server architecture, where the frontend interacts with the backend through RESTful APIs. Users can perform various operations such as event selection, ticket booking, and payment simulation. The system also maintains booking records, allowing users to track their event registrations.

Overall, this project aims to provide an efficient, user-friendly, and scalable solution for managing internal department event ticket bookings while demonstrating practical implementation of full-stack web development concepts.

1.2 Aim of the Project

The primary aim of this project is to design and develop a modern, user-friendly, and efficient web application for ticket booking of internal department events using React JS. The system is intended to provide a centralized digital platform where users can easily explore various events, access detailed information, and book tickets in a seamless manner. By replacing traditional manual registration processes, the application aims to reduce paperwork, minimize human errors, and improve the overall efficiency of event management.

Furthermore, the project focuses on implementing a full-stack architecture by integrating frontend and backend technologies. The frontend is developed using React JS to create a dynamic and responsive user interface, while the backend is built using Node.js and Express.js to handle server-side operations and API communication. MongoDB Atlas is used as a cloud-based database to store and manage user and booking data securely and efficiently.

Another important aim of this project is to demonstrate the practical application of RESTful APIs for smooth communication between the client and server. The system

also includes features such as event browsing, ticket booking, and payment simulation, which enhance user interaction and provide a realistic experience. Additionally, the project aims to build a scalable and flexible system that can be further extended with advanced features such as secure authentication, real-time notifications, and online payment integration.

Overall, this project aims to develop a reliable and scalable web-based solution that simplifies the process of event ticket booking within departments while providing a strong foundation for future enhancements and real-world applications.

1.3 Scope of the Project

The scope of this project focuses on the design and development of a web-based application for ticket booking of internal department events. The system is intended to provide a digital platform where users can view available events, access detailed information, and book tickets in an efficient and user-friendly manner. The application is limited to internal department use, such as academic events, workshops, seminars, and technical or cultural programs.

The project covers the implementation of a complete frontend using React JS, which ensures a responsive and interactive user interface. It also includes the development of a backend using Node.js and Express.js to manage server-side operations and handle API requests. MongoDB Atlas is used as a cloud-based database to store booking details and user-related information securely.

The application supports essential functionalities such as event listing, event details viewing, ticket booking, and payment simulation. It also includes basic user authentication features and maintains records of booked tickets for users. The system follows a client-server architecture and uses RESTful APIs to enable smooth communication between the frontend and backend.

However, the scope of the project is limited to a prototype-level implementation. Advanced features such as real-time notifications, integration with external payment gateways, role-based access control, and large-scale deployment are not included in the current version. These features can be considered as future enhancements.

Overall, the project demonstrates the practical implementation of a full-stack web application and provides a scalable foundation that can be extended for real-world event management systems.

1.4 Framework

The framework of the proposed system is based on a full-stack web development approach that integrates both frontend and backend technologies to deliver a complete event ticket booking solution. The application is developed using a modern technology stack that ensures scalability, flexibility, and efficient performance.

The frontend of the system is built using React JS, which is a popular JavaScript library for creating dynamic and responsive user interfaces. It allows the development of reusable components and provides a smooth user experience through efficient state management and rendering techniques. The user interface is designed to be intuitive and responsive across different devices.

The backend framework is developed using Node.js and Express.js. Node.js provides a runtime environment for executing JavaScript on the server side, while Express.js simplifies the process of building RESTful APIs and handling HTTP requests and responses. The backend is responsible for managing application logic, processing user requests, and handling communication between the frontend and the database.

MongoDB Atlas is used as the database framework for storing and managing application data. It is a cloud-based NoSQL database that stores data in a flexible

JSON-like format, making it suitable for handling dynamic data structures. Mongoose is used as an Object Data Modeling (ODM) library to interact with MongoDB, enabling schema definition and data validation.

The overall framework follows a client-server architecture, where the frontend communicates with the backend through RESTful APIs using HTTP methods such as GET and POST. This modular structure improves maintainability, scalability, and ease of development. The framework ensures that each component of the system operates independently while maintaining seamless integration across the application.

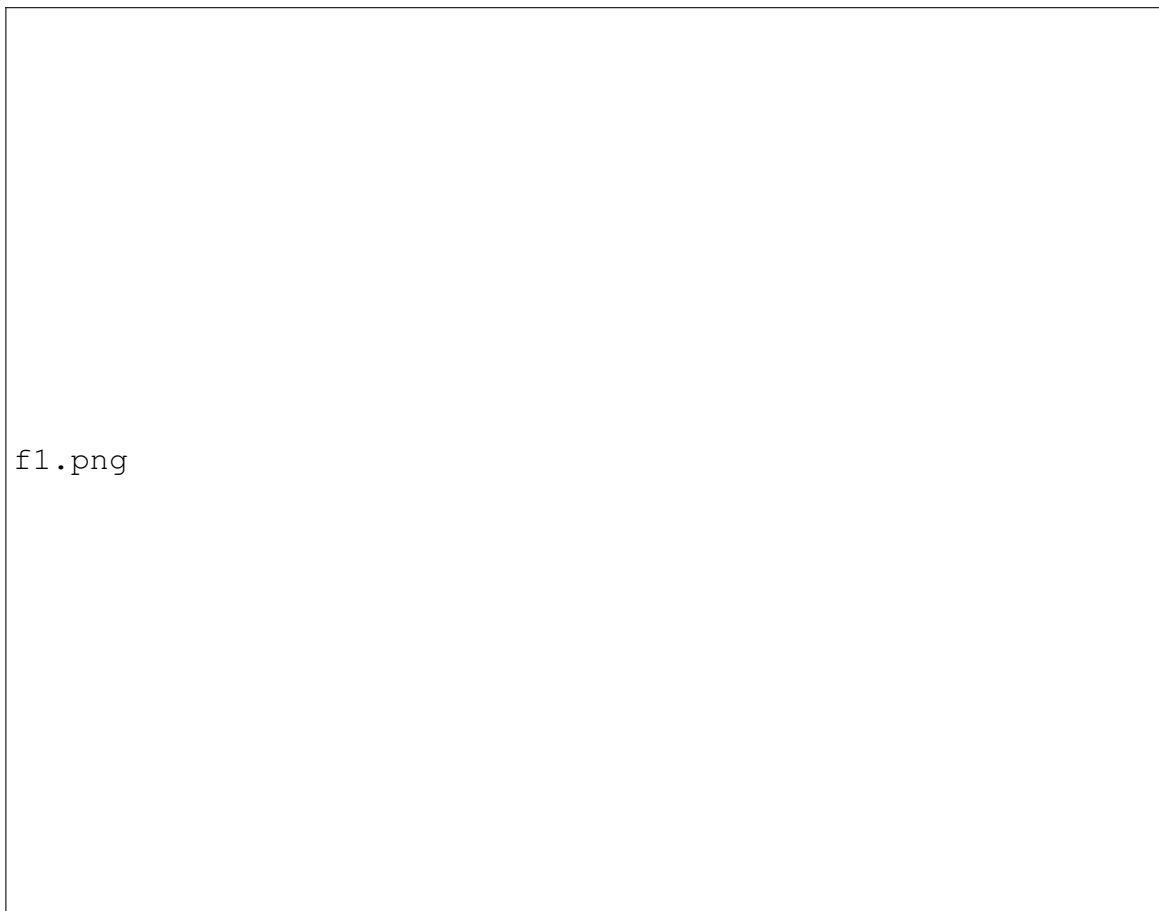


Figure 1.1: Framework Diagram

Chapter 2

SURVEY OF SIMILAR APPLICATION IN MARKET

In the current digital era, several online platforms and applications provide event ticket booking and management services. These applications have transformed the traditional ticketing process by offering convenience, real-time access, and efficient management of events.

One of the most popular platforms is BookMyShow, which allows users to discover and book tickets for movies, concerts, sports events, and live shows. It provides a wide range of event listings, multiple payment options, and user-friendly features, making it one of the leading ticket booking platforms in India ([mepass.in][1]). Another widely used platform is Paytm Insider (now integrated with District by Zomato), which enables users to explore events, book tickets, and receive real-time updates about ongoing activities and entertainment options ([Google Play][2]).

Applications such as AllEvents and EventBookings also offer comprehensive solutions for discovering and booking tickets for various events including concerts, workshops, and festivals. These platforms provide features like instant booking, event discovery, and digital ticket storage, enhancing user convenience ([Google Play][3]). Similarly, platforms like Ticket Fairy and Hytix focus on providing ad-

vanced ticketing solutions for event organizers, including real-time reporting, marketing tools, and easy event management features ([Hytix Ticketing -][4]).

In addition, modern ticketing systems emphasize features such as online payment integration, customizable registration forms, and analytics for tracking ticket sales and user engagement. These features help improve the overall efficiency and effectiveness of event management systems ([Zoho][5]).

Despite the availability of these platforms, most existing applications are designed for large-scale public events and commercial use. There is limited focus on simplified, customized solutions for internal department events within educational institutions. Therefore, this project aims to develop a dedicated web application that addresses these specific needs by providing a lightweight, user-friendly, and efficient ticket booking system tailored for departmental use.

Table 2.1: Comparative Analysis of Market Alternatives

Feature	BookMyShow	Eventbrite	Townscript	Your Project
Event Creation	Yes	Yes	Yes	Yes
Ticket Booking	Yes	Yes	Yes	Yes
Internal Department Events	No	No	No	Yes
User Registration/Login	Yes	Yes	Yes	Yes
Online Payment Integration	Yes	Yes	Yes	No / Planned
E-Ticket	Yes	Yes	Yes	Yes
Seat Selection	Yes	Limited	Limited	Yes
Event Analytics	Yes	Yes	Yes	Basic
Direct Communication (Admin-User)	No	Limited	No	Yes

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

The frontend development environment for the proposed application was set up using modern web development tools and technologies to ensure efficiency, scalability, and ease of development. The primary framework used for building the user interface is React JS, which enables the creation of dynamic and reusable components.

Initially, Node.js was installed to provide the runtime environment required for running JavaScript applications. The Node Package Manager (npm) was used to install necessary dependencies and manage project packages. The React application was created using tools such as Create React App, which provides a pre-configured setup for developing React-based applications.

A code editor, such as Visual Studio Code, was used for writing and managing the source code due to its powerful features like extensions, debugging support, and integrated terminal. Additional libraries such as React Router DOM were installed to handle navigation between different pages, and Axios was used to enable communication between the frontend and backend through API calls.

For styling, Tailwind CSS (or standard CSS) was utilized to design a responsive

and visually appealing user interface. The development server was run locally to test and debug the application during development. This setup ensured a smooth workflow and efficient development of the frontend components.

3.1.2 Structure of the Project

The project follows a modular and well-organized structure to ensure maintainability, scalability, and ease of development. The application is divided into frontend and backend components, each with its own organized directory structure.

The frontend is developed using React JS and follows a component-based architecture. The main folders include components, pages, data, routes, context, and utilities. The components folder contains reusable UI elements such as Navbar, Event-Card, and Footer. The pages folder includes different application views such as Home, Events Listing, Event Details, Booking, Payment, and Dashboard. The data folder stores static data files used during development, while the routes folder manages navigation using React Router. The context folder is used for state management, and the utils folder contains helper functions.

The backend is developed using Node.js and Express.js, and it follows a structured approach with separate folders for routes, controllers, and models. The server.js file acts as the entry point of the backend application. The routes folder defines API endpoints, the controllers folder contains the business logic, and the models folder defines the database schemas using Mongoose.

This structured organization improves code readability, simplifies debugging, and allows developers to easily add new features or modify existing functionality. It also promotes separation of concerns, where each part of the application handles a specific responsibility.

3.1.3 Execution Procedure

The execution of the project involves a series of steps starting from setting up the

development environment to running both frontend and backend applications successfully. The procedure ensures proper integration between different components of the system.

Initially, the development environment is prepared by installing Node.js and a code editor such as Visual Studio Code. The frontend application is then set up using React JS. Required dependencies such as React Router DOM and Axios are installed using the Node Package Manager (npm). The frontend server is started using the command `npm start`, which runs the application on a local development server.

Next, the backend setup is performed using Node.js and Express.js. A separate backend folder is created, and necessary packages such as Express, CORS, Mongoose, and Nodemon are installed. The server is configured in the `server.js` file, where API routes are defined to handle requests such as fetching events and processing ticket bookings. The backend server is started using the command `npm run nodemon server.js`.

For database integration, MongoDB Atlas is used. A cluster is created, and a connection string is obtained. The backend is connected to the database using Mongoose and environment variables. Once connected, booking data is stored and retrieved from the database.

After setting up both frontend and backend, API integration is performed. The frontend communicates with the backend using Axios to send and receive data. Features such as event listing, ticket booking, and payment simulation are tested to ensure proper functionality.

Finally, the complete system is tested by running both frontend and backend servers simultaneously. The application is accessed through a web browser, and all modules are verified for correct operation. Any errors identified during testing are debugged and resolved to ensure smooth execution of the application.

3.2 Backend Implementation

3.2.1 Environment Setup

The backend development environment for the proposed application was set up using Node.js and Express.js to handle server-side operations and API communication. Node.js was installed to provide a runtime environment for executing JavaScript on the server. The Node Package Manager (npm) was used to initialize the project and manage dependencies.

A separate backend directory was created, and the project was initialized using the `npm init -y` command. Essential packages such as Express.js were installed to build the web server and define API routes. The CORS middleware was added to enable communication between the frontend and backend running on different ports. Nodemon was installed as a development dependency to automatically restart the server whenever changes were made to the code.

The server configuration was implemented in the `server.js` file, which acts as the entry point of the backend application. Middleware functions such as `express.json()` were used to parse incoming JSON data from client requests.

For database integration, MongoDB Atlas was used as a cloud-based database service. Mongoose was installed to establish a connection between the Node.js application and MongoDB, and to define schemas for storing booking data. Environment variables were used to securely store the database connection string using a `.env` file, ensuring that sensitive information was not hardcoded into the application.

The backend server was executed using the command `npx nodemon server.js`, which starts the server and listens for incoming API requests. This setup ensured a structured, secure, and efficient backend environment for handling event data and ticket booking operations.

3.2.2 Structure of the Project

The backend of the project is organized in a structured and modular manner to ensure better maintainability, scalability, and readability of the code. The application follows a layered architecture where different functionalities are separated into distinct folders such as routes, controllers, and models.

The `server.js` file acts as the main entry point of the backend application. It is responsible for initializing the Express server, configuring middleware such as CORS and JSON parsing, and connecting to the MongoDB database using Mongoose. All API routes are registered in this file.

The `routes` folder contains files that define the API endpoints of the application. These routes handle incoming HTTP requests and direct them to the appropriate controller functions. Each route is designed to handle specific functionalities such as fetching event data or processing booking requests.

The `controllers` folder contains the business logic of the application. It processes the data received from the routes, performs necessary operations, and sends appropriate responses back to the client. This separation helps in keeping the code clean and easy to manage.

The `models` folder defines the database schemas using Mongoose. These schemas represent the structure of the data stored in MongoDB, such as booking details including user name, phone number, number of tickets, and event ID.

This structured organization improves code clarity, simplifies debugging, and allows easy integration of new features in the future.

3.2.3 Execution Procedure

The execution of the backend system is carried out through a sequence of well-defined steps to ensure proper functionality and integration with the frontend application. Initially, the backend project is set up by creating a dedicated directory and initializing it using the Node Package Manager (npm). Necessary dependencies

such as Express.js, CORS, Mongoose, and Nodemon are installed to support server creation, API handling, database connectivity, and efficient development.

The server is configured in the `server.js` file, where middleware functions like `express.json()` are used to parse incoming request data, and CORS is enabled to allow communication between the frontend and backend. The backend is then connected to the MongoDB Atlas database using Mongoose, with the connection string securely stored in a `.env` file.

API routes are defined to handle operations such as fetching event data and processing ticket bookings. These routes are linked to controller functions that implement the required business logic and send appropriate responses to the client.

The backend server is executed using the command `npx nodemon server.js`, which automatically restarts the server whenever changes are made during development. The server listens for incoming requests on a specified port, typically port 5000.

The frontend communicates with the backend using HTTP requests through Axios, exchanging data in JSON format. Booking requests sent from the frontend are processed by the backend and stored in the database.

Finally, the system is tested by running both frontend and backend servers simultaneously. All API endpoints are verified to ensure correct functionality, and any errors encountered during execution are identified and resolved to achieve smooth and reliable operation of the application.

3.3 Database Implementation

3.3.1 Database Configuration

The database for the proposed application is configured using MongoDB Atlas, which is a cloud-based NoSQL database service. It provides a flexible and scalable

environment for storing and managing application data in a secure manner. MongoDB stores data in a JSON-like format known as BSON, which allows easy integration with JavaScript-based applications.

Initially, a MongoDB Atlas account is created, and a cluster is set up using the free-tier plan. A database user is configured with appropriate credentials, and network access is enabled to allow connections from the application. A connection string is generated from the MongoDB Atlas dashboard, which is used to connect the backend application to the database.

In the backend, Mongoose is used as an Object Data Modeling (ODM) library to establish a connection between Node.js and MongoDB. The connection string is stored securely in an environment variable using a `.env` file to prevent exposure of sensitive information. The connection is initialized in the `server.js` file using the Mongoose connect method.

Once the connection is successfully established, the database is ready to store and retrieve data. Collections are automatically created when data is inserted for the first time. The database is used to store booking information such as user details, number of tickets, and event identifiers.

This configuration ensures a reliable, secure, and scalable database setup that supports efficient data management for the ticket booking application.

3.1.2 Schema Structure

The schema structure of the database is defined using Mongoose, which provides an efficient way to model and manage data in MongoDB. A schema acts as a blueprint that specifies how data is organized within the database, including field names, data types, and validation constraints. This ensures that all stored data follows a consistent and well-defined format.

In this project, a booking schema is created to store information related to event

ticket reservations. The schema includes essential fields such as user name, phone number, number of tickets, and event identifier. Each field is assigned an appropriate data type, such as String for textual data and Number for numerical values. Validation rules are also applied to ensure that mandatory fields are not left empty and that the data entered by users is accurate and reliable.

Additionally, the schema may include timestamps to automatically record the date and time when a booking is created or updated. This feature helps in tracking user activity and maintaining a history of bookings. The schema structure also supports scalability, allowing new fields to be added in the future without affecting the existing data.

The schema is used to create a model, which acts as an interface between the application and the database. Through this model, various operations such as creating, reading, updating, and deleting booking records can be performed efficiently. Whenever a user submits a booking request, the data is validated against the schema before being stored in the database, ensuring data integrity.

Overall, the schema structure plays a crucial role in maintaining consistency, reliability, and organization of the data. It simplifies database operations and enhances the overall performance of the application by ensuring that all records are stored in a structured and standardized manner.

3.1.3 Tables created

Although MongoDB is a NoSQL database and does not follow the traditional table-based structure of relational databases, it organizes data using collections and documents. In this project, collections function similarly to tables, where each collection stores related data in a structured yet flexible format. This approach allows efficient handling of dynamic and unstructured data.

The main collection used in this application is the booking collection, which

stores all information related to ticket reservations. Each document in this collection represents an individual booking and contains fields such as user name, phone number, number of tickets, event identifier, and timestamps indicating when the booking was created. This structure enables easy tracking and management of booking records.

Another important collection is the events collection, which stores details about the available events. This includes information such as event name, description, date, time, venue, and ticket price. Storing event data in a separate collection helps in dynamically displaying event information on the frontend and allows easy updates without modifying the application code.

In some cases, a users collection can also be maintained to store user-related information such as name, email, and login credentials, especially if authentication features are implemented. This allows better management of user accounts and personalization of services.

MongoDB automatically creates collections when data is inserted for the first time, eliminating the need for predefined table structures. This flexibility makes it easier to modify the data model, add new fields, or scale the application as requirements grow.

Overall, the use of collections instead of traditional tables provides a flexible, scalable, and efficient data storage mechanism. It simplifies database management and supports the dynamic nature of the event ticket booking system, ensuring smooth handling of user and event-related data.

Chapter 4

METHODOLOGIES

4.1 Project Architecture

Figure 4.1 The above diagram represents the overall architecture of the Event Ticket Booking System, which follows a three-tier client-server model consisting of the frontend, backend, and database layers.

The frontend layer is developed using React.js and acts as the user interface of the application. It allows users to perform various operations such as browsing events, viewing event details, booking tickets, managing bookings, making payments, and handling login or registration. The frontend communicates with the backend by sending HTTP requests using Axios and receives responses in JSON format.

The backend layer is implemented using Node.js and Express.js, which handle the application's business logic and API processing. The backend exposes RESTful API endpoints such as retrieving events, creating bookings, updating bookings, and deleting bookings. It also includes modules for authentication, booking management, payment handling, and validation middleware to ensure secure and proper data processing.

The database layer uses MongoDB Atlas, a cloud-based NoSQL database. It stores data in collections such as events and bookings. The events collection con-

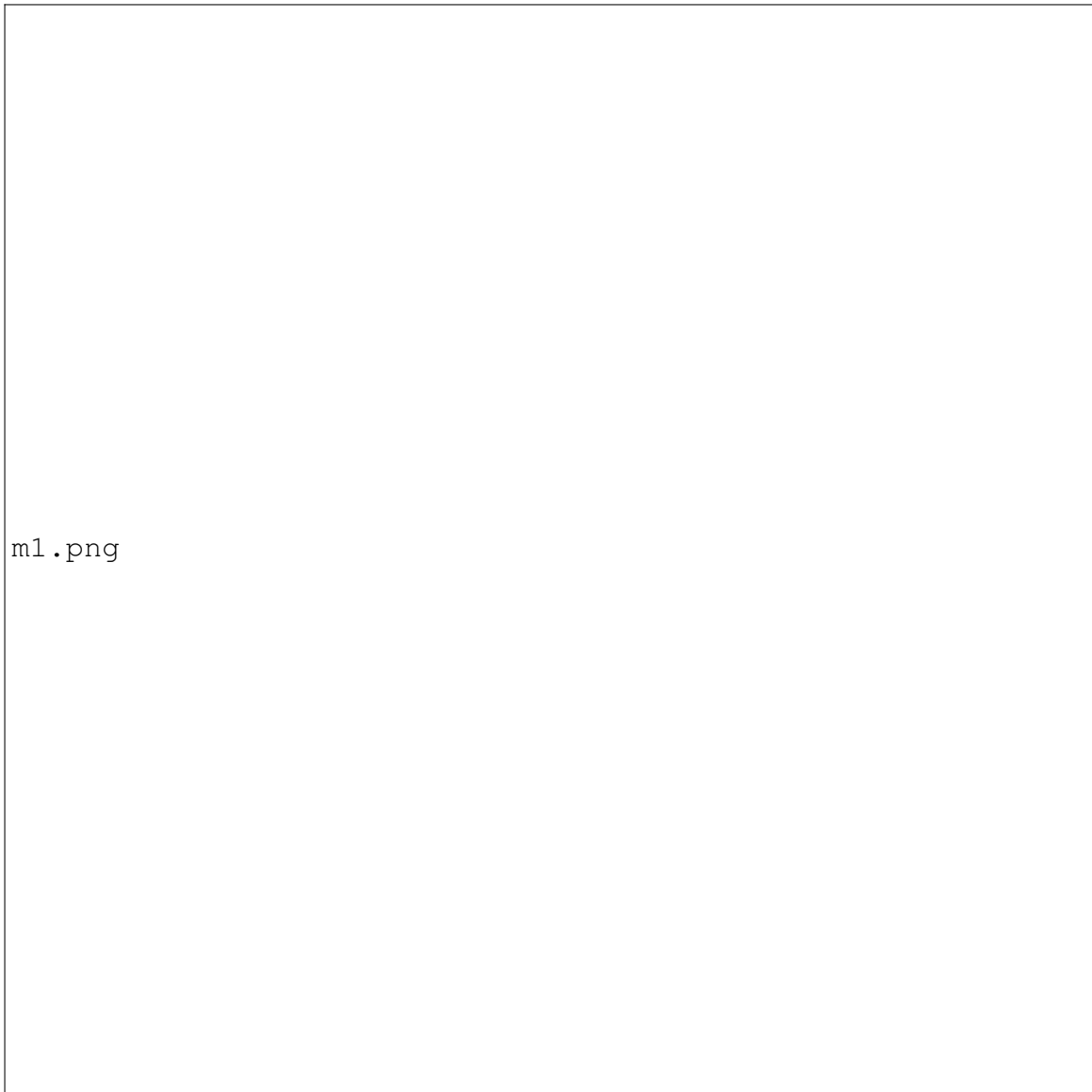


Figure 4.1: Architecture Diagram

tains details like event name, description, date, venue, price, and image, while the bookings collection stores user-related booking information such as name, phone number, number of tickets, event ID, and timestamps. The backend interacts with the database using Mongoose for efficient data operations.

Additionally, the system integrates with external services such as authentication services (JWT), payment gateways (Stripe or Razorpay), email services (Nodemailer), and cloud storage (Cloudinary). These services enhance the functionality of the system by providing secure authentication, payment processing, communication,

and file storage capabilities.

Overall, the architecture ensures smooth communication between components, efficient data handling, and scalability of the application, making it suitable for managing internal department event ticket bookings effectively.

4.2 DataFlow Diagram

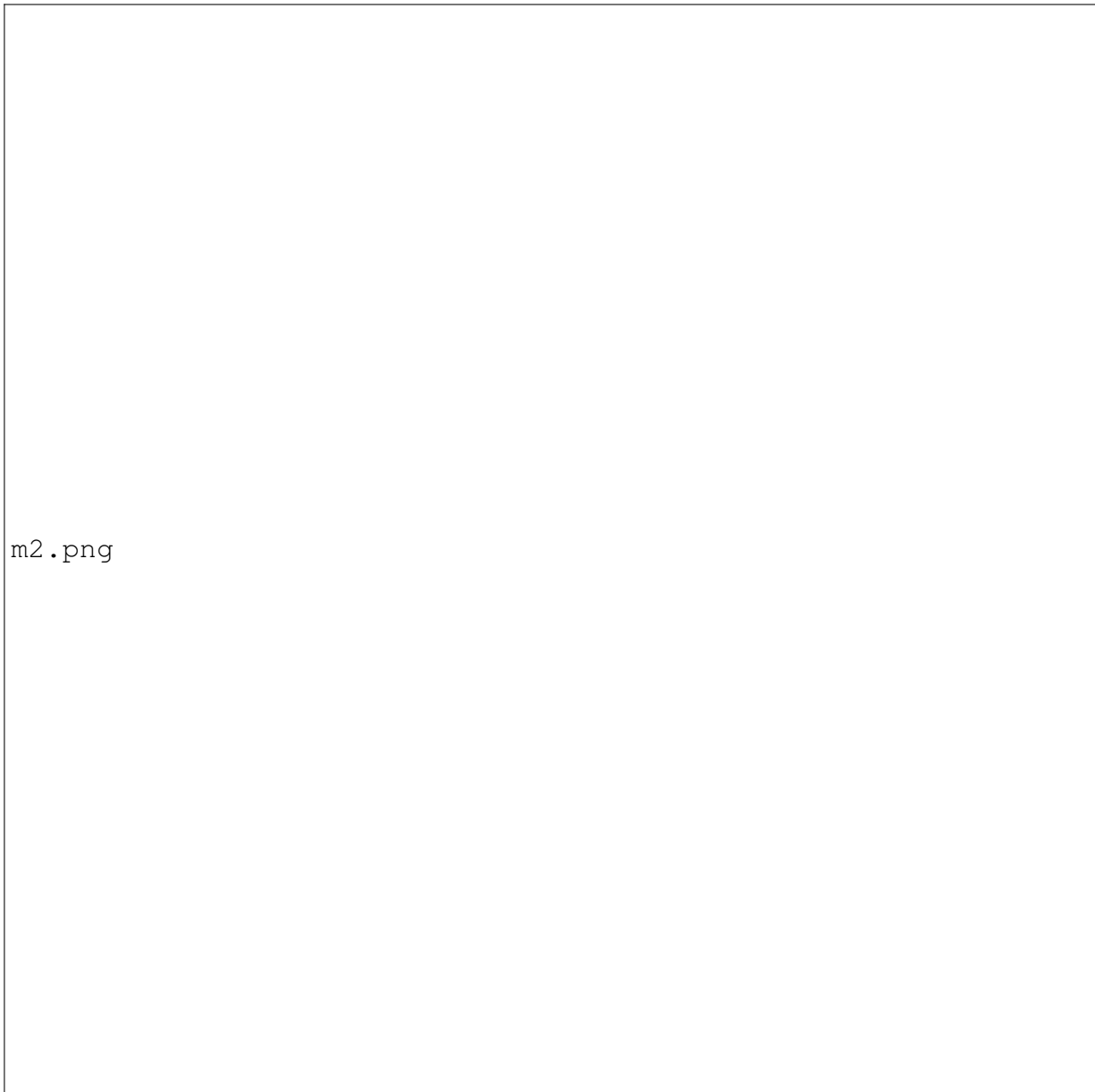


Figure 4.2: Data Flow Diagram

The above diagram represents the Level 0 Data Flow Diagram (DFD) of the

Event Ticket Booking System. It illustrates how data flows between the external user, system processes, and data storage components.

The system begins with the external entity, the user, who interacts with the application by performing actions such as registration, login, browsing events, viewing event details, booking tickets, making payments, and managing bookings. These user inputs are sent as requests to the main process, labeled as “1.0 Process User Request.”

The process “1.0 Process User Request” acts as the central unit that coordinates all system activities. It communicates with other processes such as “2.0 Manage Events,” which handles fetching and updating event-related data, and interacts with the Events Collection (D1) in the database. This ensures that users can view up-to-date event information.

For booking operations, the system uses “3.0 Manage Bookings,” which processes booking data received from the user and stores or updates it in the Bookings Collection (D2). Once a booking is made, a confirmation request is generated.

The payment functionality is handled by “4.0 Process Payment,” where payment requests are processed and payment status is generated. Upon successful payment, the system proceeds to “5.0 Generate Confirmation,” which produces the final confirmation, such as a ticket or invoice, and sends it back to the user.

The database layer consists of MongoDB Atlas, which stores all event and booking data in separate collections. Data flows between the processes and database include saving, updating, and retrieving information as needed.

Overall, the diagram clearly shows the flow of data between the user, system processes, and database, ensuring efficient handling of event management, booking, payment, and confirmation within the application.

4.3 Module 1: User Management

The User Management module is responsible for handling all activities related to user interaction and authentication within the system. It allows new users to register by providing essential details such as name, email, and password. The system validates user inputs to ensure correctness and prevent invalid data entry. Once registered, users can log in securely using their credentials.

This module also manages user sessions and ensures that only authenticated users can access certain features such as booking tickets and viewing personal booking history. It may use techniques such as token-based authentication to maintain session security. Additionally, users can view and manage their profile information, enhancing personalization and usability. Proper error handling and validation mechanisms are implemented to improve reliability and user experience.

4.4 Module 2: Event Management

The Event Management module is designed to handle all operations related to event data. It stores and manages information about different events, including event name, description, date, time, venue, and ticket pricing. This module ensures that event information is accurately maintained and updated in the database.

Users can browse a list of available events and view detailed descriptions before making a booking decision. The module supports dynamic data retrieval, allowing events to be displayed in real time from the database rather than using static content. It also enables easy addition, modification, or deletion of event details, making the system flexible and adaptable to changing requirements. This module plays a key role in providing users with up-to-date and relevant event information.

4.5 Module 3: Booking and Payment Management

The Booking and Payment Management module is the core component of the system, responsible for handling ticket reservations and payment processing. Users can select an event, specify the number of tickets, and proceed to book them through an interactive interface. The system calculates the total cost based on the number of tickets selected.

The payment functionality in this project is implemented as a simulation, where users enter the required amount to complete the transaction. Upon successful payment, the system confirms the booking and generates a ticket or confirmation message. All booking details, including user information and transaction data, are stored in the database for future reference.

This module also allows users to view their booking history and cancel bookings if required. Proper validation ensures that incorrect or incomplete booking data is not processed. Overall, this module ensures a smooth and reliable booking experience for users.

Advantages of this Project

The proposed system offers several advantages by providing a modern and efficient solution for managing internal department event bookings. It reduces manual effort, improves accuracy, and enhances user convenience through a digital platform.

a) User-Friendly Interface: The application is designed with a clean and intuitive interface that allows users to easily navigate through different sections such as events, booking, and payment. This improves accessibility and reduces the learning curve for new users.

b) Efficient Data Management: The use of MongoDB enables flexible and efficient storage of large amounts of data. It allows quick retrieval and updating of event and booking information, ensuring smooth system performance.

c) Scalability and Flexibility: The system is developed using modern technologies like React, Node.js, and Express.js, which support scalability. New features such as real-time notifications or online payments can be added easily in the future.

d) Reduced Manual Work: The system automates the process of event registration and ticket booking, reducing the need for manual record-keeping and minimizing human errors.

e) Centralized Management: All event and booking data are stored in a centralized database, making it easy to manage, monitor, and retrieve information when needed.

f) Improved Accuracy: Automated validation and processing reduce the chances of errors in booking and payment details, ensuring reliable system operation.

Disadvantages

- The payment system is only a simulation and does not support real-time online transactions.
- The application depends on internet connectivity, which may affect accessibility during network issues.
- Limited security features may lead to risks related to data privacy and unauthorized access.
- The system is designed mainly for internal department use and may not handle large-scale public events efficiently.
- Lack of advanced features such as real-time notifications and multi-user role management.
- Requires continuous maintenance and updates to ensure proper functionality and security.

- No integration with external systems such as payment gateways or email verification services in the current version.

Chapter 5

RESULTS AND DISCUSSIONS

The Ticket Booking System for an Internal Department Event is developed as a comprehensive platform that streamlines event management and ticket booking processes within an organization. The system integrates multiple functional modules to provide a seamless and efficient user experience for both users and administrators.

5.1 System Modules and Features

The implemented system consists of the following key modules:

- **Landing Page Interface:** Provides an overview of available events and navigation options for users.
- **User Authentication Module:** Handles user registration, login, and secure authentication.
- **Event Listing and Management Module:** Displays event details and allows administrators to create, update, or delete events.
- **Ticket Booking System:** Enables users to book tickets by selecting events and entering required details.

- **Seat Selection Module:** Allows users to choose seats visually and ensures availability before booking.
- **Payment Processing Module:** Manages payment transactions and confirms successful bookings (if applicable).
- **E-Ticket and QR Code Generation:** Generates digital tickets with QR codes for easy verification.
- **User Dashboard (My Tickets):** Allows users to view their booking history and download tickets.
- **Admin Dashboard:** Provides administrative control for managing events, users, and bookings.
- **Notification System:** Sends alerts via email or SMS for booking confirmations and updates.
- **Reporting and Analytics Module:** Generates reports and insights related to bookings and events.
- **Database Layer:** Stores all system data including users, events, tickets, and transactions.

5.2 System Performance and Discussion

The system follows a modular and layered architecture that ensures scalability, secure communication, and efficient handling of event-related operations. It enables smooth interaction between users and administrators while maintaining data integrity and real-time updates throughout the booking process.

The performance of the system has been evaluated based on responsiveness, data handling capability, and user interaction efficiency. The application provides quick

response times during event browsing, ticket booking, and data retrieval operations. The integration between the frontend and backend ensures seamless data flow without noticeable delays.

The system effectively manages multiple user requests and maintains consistency in database operations. Proper validation mechanisms are implemented to prevent incorrect data entry and ensure reliability. The use of structured queries for storing and retrieving data enhances performance and accuracy.

In terms of usability, the interface is designed to be simple and intuitive, allowing users to easily navigate through different features such as viewing events, booking tickets, and accessing booking details. The administrator module provides efficient control over event management, including adding, updating, and monitoring event information.

The system is flexible and can be further enhanced by integrating advanced features such as online payment gateways and notification services. It also provides a strong foundation for future expansion to support larger scale event management systems.

The results demonstrate that the system significantly reduces manual effort, minimizes human errors, and improves overall operational efficiency. Automated processes such as data storage, retrieval, and validation contribute to better system performance. Overall, the application provides a reliable, scalable, and user-friendly solution for managing internal department events.

5.3 System Interface Visualizations

5.3.1 Landing Page Interface

The landing page (Fig. 5.1) provides an overview of the system and introduces users to available events. It presents a clean and interactive interface

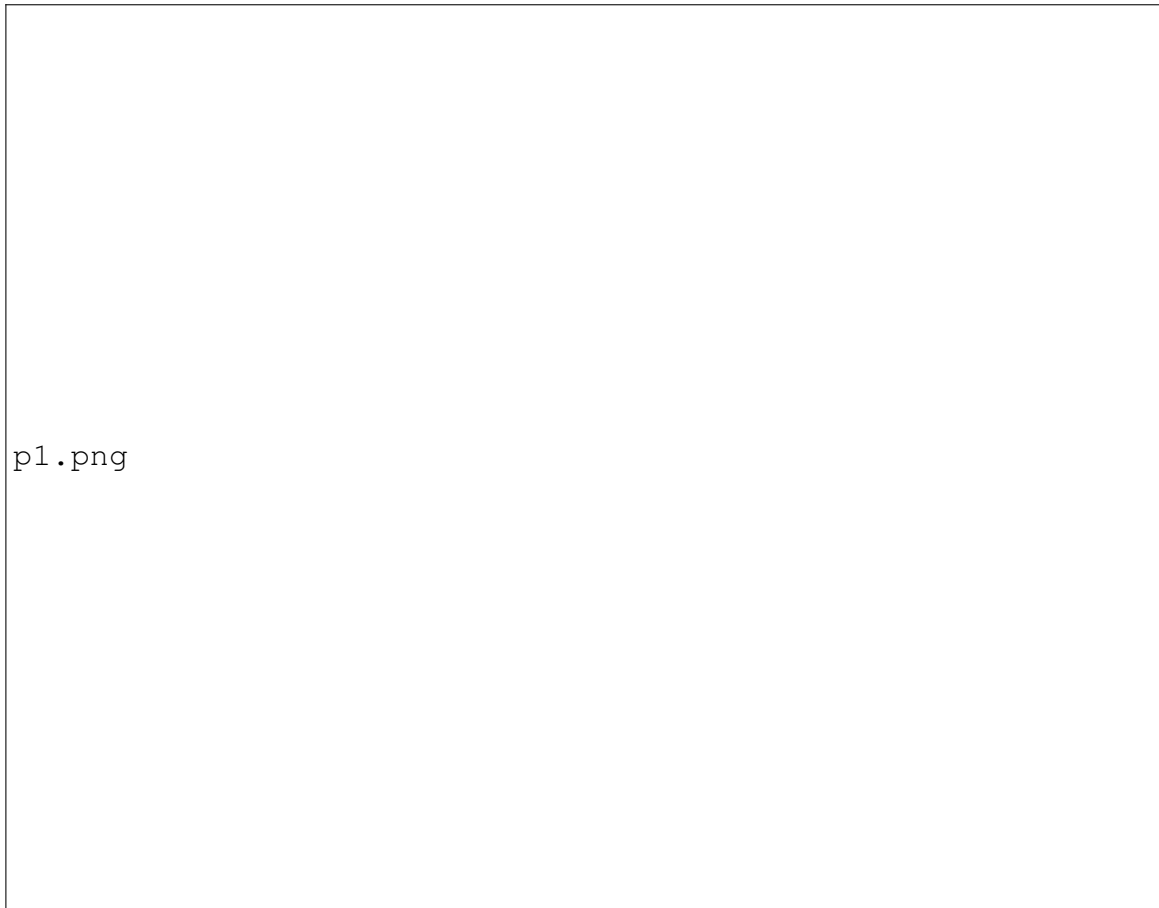


Figure 5.1: Framework Diagram

5.3.2 User Authentication:

The user authentication module (Fig. 5.2) allows users to securely log in using valid credentials. It ensures authorized access and maintains data security through proper validation mechanisms.

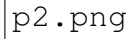
A large rectangular box representing the user authentication interface, which is currently blank.

Figure 5.2: User Authentication

5.3.3 Event Listing Management

The event listing module (Fig. 5.3) displays all available events with details such as title, description, and booking options. It helps users efficiently explore and select events of their interest.

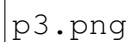
A large rectangular box representing the event listing management interface, which is currently blank.

Figure 5.3: Event Listing Management

5.3.4 Ticket Booking System

The ticket booking system (Fig. 5.4) allows users to select seats and proceed with the booking process. It ensures a smooth and error-free experience through a structured and guided workflow.

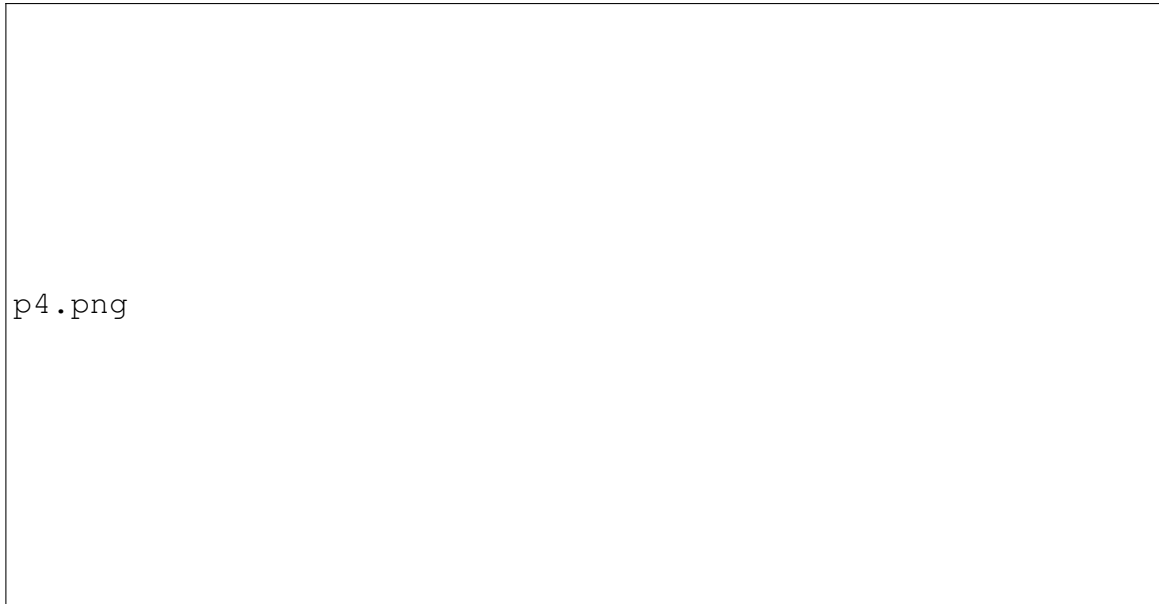


Figure 5.4: Framework Diagram

5.3.5 Booking Details

The booking details module (Fig. 5.5) collects necessary user information required to complete the reservation. It captures details such as name, contact information, and booking preferences. The module ensures that all mandatory fields are properly filled before submission. Validation checks are applied to maintain accuracy and avoid incorrect entries. This helps in ensuring a smooth and reliable booking process.

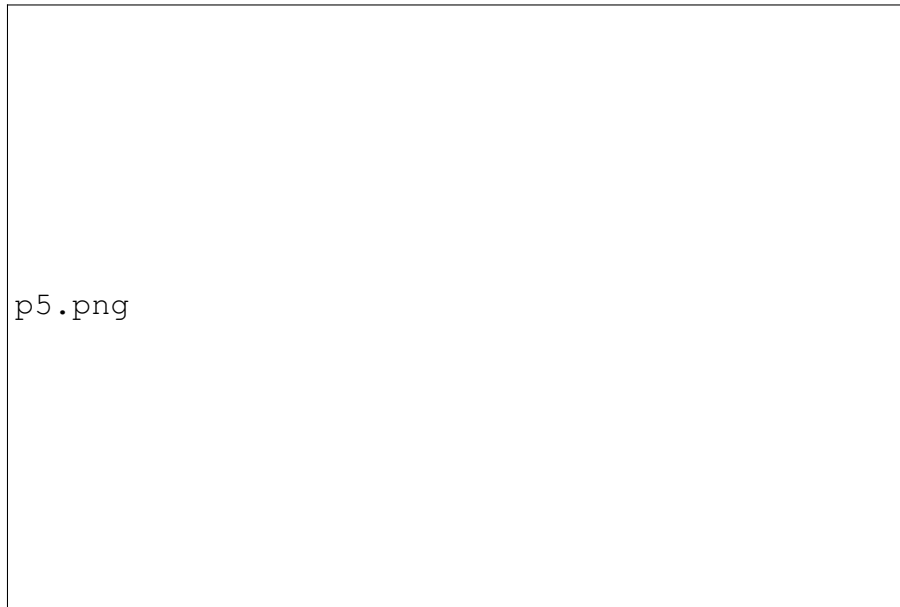


Figure 5.5: Framework Diagram

5.3.6 User Dashboard

The user dashboard (Fig. 5.9) provides a centralized view of all user activities, including booked tickets and event details.

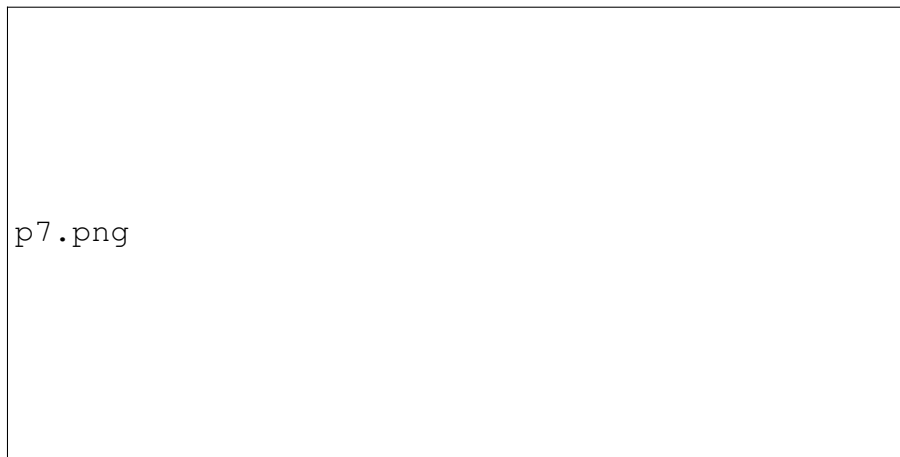


Figure 5.6: User Dashboard

5.3.7 Ticket Generation

The ticket generation module (Fig. 5.7) produces a digital ticket after successful booking. The generated ticket contains all relevant information such as event details and booking confirmation. It Provides users with a clear and verifiable proof of their reservation.

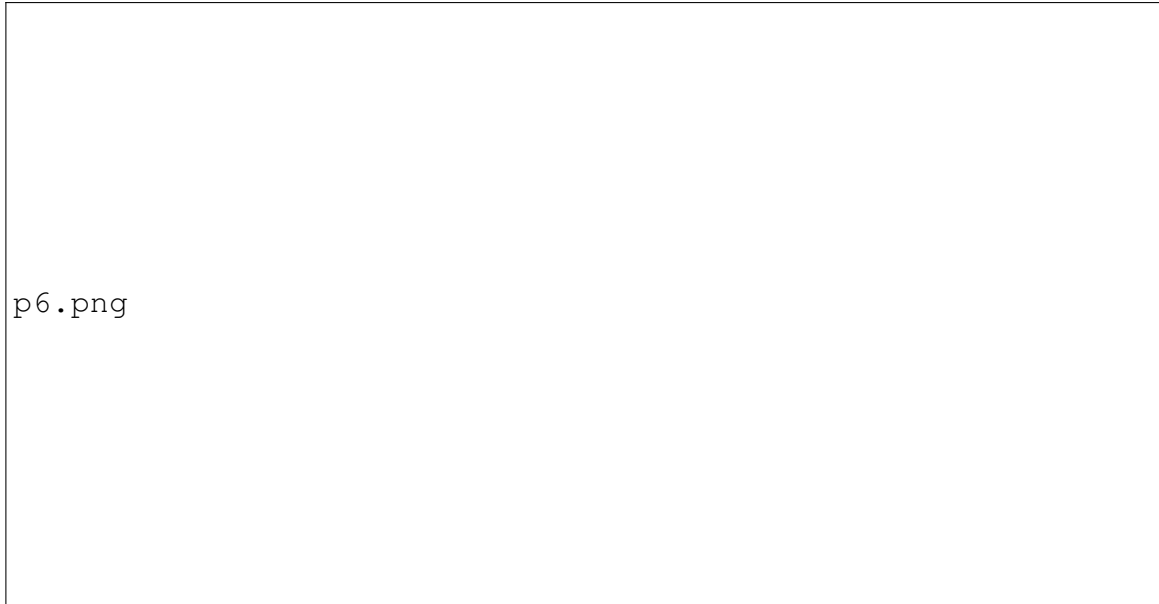


Figure 5.7: Ticket

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The Event Ticket Booking System developed in this project provides an efficient and user-friendly solution for managing internal department events. The system successfully replaces traditional manual methods with a modern web-based approach, improving speed, accuracy, and accessibility.

The frontend, developed using React JS, offers a dynamic and responsive user interface, while the backend built with Node.js and Express.js efficiently handles application logic and API communication. MongoDB Atlas is used for secure and scalable data storage, ensuring proper management of event and booking information.

The system enables users to browse events, book tickets, and manage their bookings with ease. The integration of different modules such as user management, event management, and booking with payment simulation ensures smooth functioning of the application.

Overall, the project demonstrates the effective use of modern web technologies to build a full-stack application. It simplifies event management processes and pro-

vides a foundation for future enhancements such as real-time payment integration, advanced security features, and notification systems.

6.2 Future Enhancements

The current system provides a functional platform for event ticket booking; however, several enhancements can be implemented to improve its efficiency and usability. One major enhancement is the integration of real-time online payment gateways such as Razorpay or Stripe, which would enable secure and actual transactions instead of simulated payments.

Another improvement would be the implementation of advanced authentication mechanisms, including JWT-based authentication and role-based access control for users and administrators. This would enhance the security and allow better management of the system.

The system can also be extended by adding real-time notifications using email or SMS to inform users about booking confirmations, event updates, and reminders. Integration of push notifications would further enhance user engagement.

In addition, features such as seat selection, ticket generation in PDF format, QR code-based entry verification, and cancellation/refund mechanisms can be included. The application can also be scaled to support large-scale public events with improved performance optimization.

Finally, deploying the application on cloud platforms and implementing responsive mobile support or a dedicated mobile app would further increase accessibility and usability.

Chapter 7

ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

7.1 Mapping of the Project to Complex Engineering Problem (CEP)

The Event Ticket Booking System developed in this project can be categorized as a complex engineering problem due to its requirement for integrating multiple technologies and addressing diverse system constraints. The project demands in-depth knowledge of frontend and backend technologies such as React JS, Node.js, and database systems, along with the application of validation techniques and user interface design principles.

The system involves handling conflicting requirements, where usability must be balanced with security and data validation. It requires careful analysis and design of components, efficient state management, and dynamic updates to ensure real-time interaction during ticket booking. The project also introduces partially new challenges such as real-time booking updates and seamless user experience.

Furthermore, the implementation involves custom logic for booking processes, validation mechanisms, and data handling, as standard solutions are not sufficient

for the specific requirements of the system. The project addresses the needs of multiple stakeholders, including students, faculty, and event organizers, ensuring smooth event management and accessibility.

Finally, the system demonstrates interdependence by integrating various modules such as user interface, state management, validation, and backend services into a unified application, making it a comprehensive example of a complex engineering problem.

Table 7.1: Mapping of the Project to Complex Engineering Problem (CEP)

Level	Attribute	Characteristics	WKs	Justification
WP1	Depth of Knowledge	Requires in-depth engineering knowledge	WK4, WK5	Knowledge of React JS, hooks, validation, and UI design.
WP2	Conflicting Requirements	Technical and non-technical conflicts	WK4, WK5	Balances usability with booking constraints and validation.
WP3	Depth of Analysis	Analytical and design thinking	WK3, WK4	Component design, state handling, and dynamic updates.
WP4	Familiarity	Partially new problems	WK4	Real-time booking and ticket updates.
WP5	Applicable Codes	Limited standard solutions	WK6	Custom booking and validation logic.
WP6	Stakeholder Needs	Multiple stakeholders	WK5, WK6	Students, faculty, and event organizers.
WP7	Interdependence	System-level integration	WK3, WK5	UI, state management, and validation integration.

7.2 Mapping of the Project to Complex Engineering Activities (CEA)

Table 7.2: Mapping of the Project to Complex Engineering Activities (CEA)

EA No.	Attribute	Characteristics	Justification
EA1	Range of Resources	Use of technologies	React JS, HTML, CSS.
EA2	Level of Interactions	Complex module interaction	Event display, booking, and validation process.
EA3	Innovation	Modern technologies	Hooks and conditional rendering.
EA4	Consequences	Impact on users	Simplifies event booking process.
EA5	Familiarity	Beyond standard	Dynamic real-time system.

The Event Ticket Booking System developed in this project involves various complex engineering activities that require the effective utilization of modern web technologies and the integration of multiple system components. The project makes use of technologies such as React JS, HTML, CSS, and backend frameworks to build a complete full-stack application. These technologies contribute to designing an interactive user interface, managing user inputs, and efficiently handling backend operations.

The system exhibits a high level of interaction among different modules, including event display, ticket booking, user authentication, and validation. These modules are interconnected to ensure smooth data flow and real-time updates, thereby enhancing the overall functionality of the system. Additionally, the project incorporates modern development practices such as React hooks and conditional rendering, which improve performance and provide a better user experience.

7.3 SDG Mapping (CEP Section)

Table 7.3: SDG Mapping for the Project

SDG No.	SDG Title	Relevance
SDG 4	Quality Education	Supports academic events and learning activities.
SDG 9	Innovation	Promotes development of modern React-based systems.
SDG 10	Reduced Inequalities	Accessible to all users irrespective of background.
SDG 11	Sustainable Communities	Encourages organized and efficient event management.

References

- [1] BookMyShow – Online Ticket Booking Platform. <https://in.bookmyshow.com>
- [2] Paytm Insider / District – Events and Ticket Booking Application.
<https://insider.in>
- [3] AllEvents – Discover Events Platform. <https://allevents.in>
- [4] Hytix – Event Ticketing Platform. <https://www.hytix.com>
- [5] Zoho Backstage – Event Management and Ticketing Software.
<https://www.zoho.com/backstage>
- [6] Eventbrite – Global Event Ticketing Platform. <https://www.eventbrite.com>
- [7] Ticketmaster – Online Ticket Sales and Distribution.
<https://www.ticketmaster.com>
- [8] Meetup – Event Organization and Community Platform.
<https://www.meetup.com>
- [9] Brown Paper Tickets – Online Event Registration Platform.
<https://www.brownpapertickets.com>
- [10] Eventzilla – Event Registration and Ticketing Software.
<https://www.eventzilla.net>